

ARCHITECTURE DIAGRAM

Personal Productivity & Task Execution Agent

Prepared by: Hooria · AI Summer Internship — NLP & AI Agents Track · July 2026

How to Read This Diagram

The center column (User → Frontend → Agent API → Agent State → Tool Registry → Task/Notes/Planning Tools → Database → Execution Logs) is the main request path every request travels down, exactly as specified. Solid arrows are the main flow. Two additions sit directly on that path because they change what happens on nearly every request: a Gate that checks whether the selected tool is gated, and a Human Approval step that the flow visibly detours through before any write action executes. Dashed and dotted lines mark cross-cutting concerns — Session Memory, Error Handling, and Observability — that don't sit on the main path but touch several parts of it at once, plus the Model Provider that sits behind the LLM box.

The full diagram is on the next page in landscape orientation for readability. A detailed explanation of every single labeled component follows after it, in the order requested.

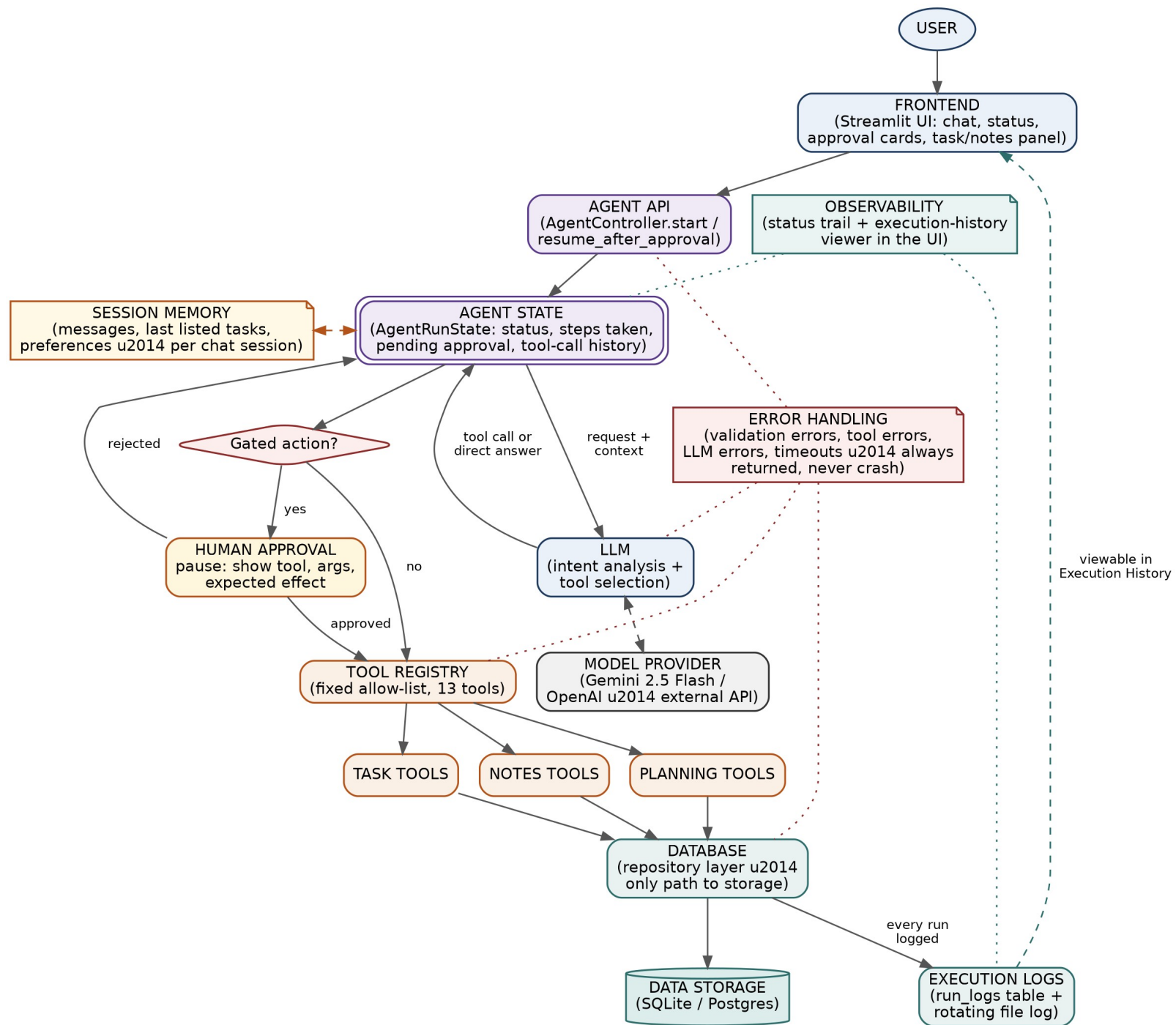


Figure 1 — Full system architecture: main request path, the human approval gate, and the three cross-cutting concerns (session memory, error handling, observability).

Component-by-Component Explanation

1. User

The person interacting with the agent — in this build, a single local user (no multi-tenant login). The user only ever interacts through the Frontend; they never call the Agent API, a tool, or the database directly. Everything the user can do is mediated by the layers below, which is what makes approval gating and validation actually effective — there's no side channel that bypasses them.

2. Frontend

The Streamlit UI (`app/main.py`). Its job is purely presentation and wiring: render the chat history, show the agent's live status, render the approval card with Approve/Reject buttons, show the task/notes panels, and show the execution-history viewer. It contains no business logic — every decision (which tool to call, whether approval is required, how to validate input) happens in the layers below it. This separation means the same agent logic could be put behind a completely different frontend (a FastAPI + React app, for example) without changing anything except this one layer.

3. Agent API

The programmatic entry point into the agent: `AgentController.start()` for a new request and `AgentController.resume_after_approval()` for continuing a paused one (`app/agent/controller.py`). This is the boundary between “UI concerns” and “agent concerns” — the Frontend calls exactly these two methods and nothing else, passing in the user's text and getting back a state object it can render. Because this boundary is a plain Python API (not tied to Streamlit), it is also the shape a future FastAPI wrapper would expose over HTTP.

4. Agent State

The per-request state object, `AgentRunState`: which step the run is on, how many steps have been taken, the current status (thinking / selecting a tool / waiting for approval / executing / responding / error), the most recent tool result, and the history of tool-call signatures used to detect loops. It is created fresh for every request and discarded once that request is fully answered. It is the hub of the whole diagram — every other box either feeds it information or receives instructions from it, which is why it sits at the center of the main path.

5. LLM

The reasoning step: given the request, the conversation so far, and the schemas of every available tool, the model decides whether to answer directly or to call a specific tool with specific arguments (implemented in `app/agent/nodes.py::analyze_and_select`, called once per loop iteration). This is where intent analysis and tool selection actually happen; it is a single native function-calling request to whichever provider is configured, not a separate hand-written classifier.

6. Tool Registry

A fixed, code-level list of every tool the agent is allowed to call, built once at startup (`app/tools/registry.py`). The LLM can only ever select from names in this registry — there is no mechanism for it to invent a new tool or call arbitrary code, which is what makes the tool surface auditable and safe to reason about.

7. Task Tools

The six tools that operate on the task store: `create_task`, `list_tasks`, `update_task`, `complete_task`, `detect_overdue_tasks`, and `create_reminder` (`app/tools/task_tools.py`). Each validates its own input against a Pydantic schema and goes through the Database layer for every read or write — none of them touch storage directly.

8. Notes Tools

The three tools that operate on the notes store: `search_notes`, `save_note`, and `summarize_notes` (`app/tools/note_tools.py`). `search_notes` and `summarize_notes` are read-only; `save_note` is a low-risk write (a single new record, not modifying anything existing) and is not gated.

9. Planning Tools

The four tools that reason over tasks/notes rather than just fetching them: `extract_meeting_actions`, `generate_work_plan`, `draft_followup_email`, and `generate_weekly_report` (`app/tools/planning_tools.py`). Two of these (extraction and email drafting) are themselves LLM-backed — they make their own call to the Model Provider with a JSON-schema-constrained prompt, separate from the tool-selection call made by the LLM box above.

10. Database

The repository layer (`app/database/repository.py`) — the only module in the entire codebase allowed to use the ORM directly. Every tool that needs to read or write task/note data calls into `TaskRepository` or `NoteRepository` here, never the ORM models directly. This single choke point is what makes it possible to say with confidence that no tool can corrupt data through a path that bypasses validation or logging.

11. Execution Logs

A structured record of every run — request text, model used, every tool call with its arguments/result/duration/approval status, any errors, and total duration — written both to the `run_logs` database table and a rotating file log (`app/agent_logging/run_logger.py`). It exists purely for accountability/debugging, not to influence the agent's own future decisions, and deliberately excludes API keys and raw model reasoning.

12. Human Approval

A hard pause point, enforced in code (the `GATED_TOOLS` set in `app/agent/controller.py`) rather than only requested in the prompt: before `update_task`, `complete_task`, `create_reminder`, `draft_followup_email`, or creating more than one task in a run, the agent stops and shows the proposed tool, its exact arguments, and the expected effect, and waits for an explicit Approve or Reject. Approving resumes the exact same run and executes the tool; rejecting records the rejection and returns control to the user with nothing changed. Because the check happens before the Tool Registry dispatches to any tool, no gated tool can execute without passing through this box first — regardless of how the LLM was prompted or phrased its intent.

13. Error Handling

Not a single component but a discipline applied at every layer: invalid tool arguments are caught by Pydantic validation before a tool's logic runs; a tool's own internal exceptions are caught by `BaseTool.execute()` and converted into `ToolResult(success=False, error=...)`; LLM/API failures are wrapped as `LLMError` with a plain-language message; unknown tool names, unknown record IDs, and database failures are all returned as a handled result rather than an unhandled exception. The controller never lets a lower-layer crash propagate to the user as a stack trace — the worst case is a clear “I couldn't complete that: ...” message.

14. Session Memory

`SessionMemory` (`app/agent/state.py`), held for the life of the chat session: recent messages, the most recently listed tasks (so “the second one” resolves to a real `task_id`), the latest output of each tool, and any preferences stated during the session. It is fed into Agent State at the start of every request and updated at the end of every request — the mechanism that makes multi-turn follow-ups work at all.

15. Model Provider

The actual external LLM — Gemini 2.5 Flash by default, or an OpenAI model if configured — reached through `app/services/llm_service.py`, which normalizes both providers' function-calling and JSON-mode APIs behind one interface (`chat_with_tools`, `generate_json`, `generate_text`). Neither the LLM box's decision logic nor any tool needs to know which provider is active; swapping `LLM_PROVIDER` in `.env` is the only change required.

16. Data Storage

The actual persistence engine behind the Database/repository layer — SQLite by default (`DATABASE_URL=sqlite:///./data/productivity_agent.db`), swappable for Postgres/Supabase with a one-line

configuration change since all access goes through SQLAlchemy. This is where tasks, notes, and the execution-log table physically live on disk.

17. Observability

The ability to see what the agent actually did, after the fact: the Frontend's Execution History panel reads directly from Execution Logs to show, per run, every tool called, its duration, its success/failure, and its approval status. Combined with the rotating file log, this is what makes a run debuggable without re-reading source code — the log is the audit trail Requirement 10 asks for.